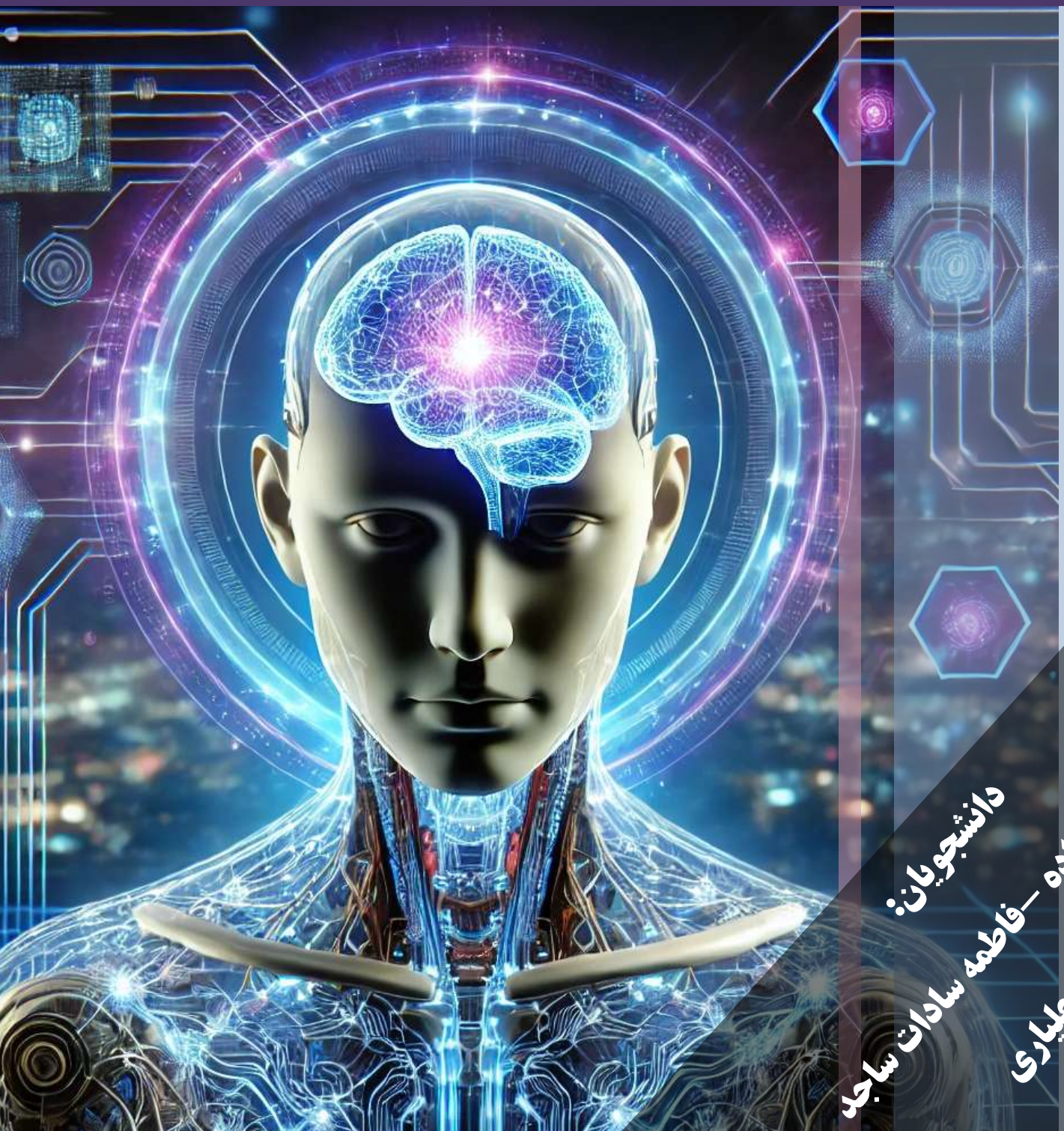
دانشگاه
خواجه نصیرالدین طوسی
K. N. Toosi University
of Technology

ترم ۱۴۰۳-۱۴۰۴

تکلیف اول هوش مصنوعی مقدماتی



دانشجویان:
آیلار رحیم زاده -فاطمه سادات ساجد
دبیر: دکتر علیاری

فهرست

مقداری جبر خطی..... ۳

پردازش داده ۷

CWRU Dataset ۷

دریافت داده ۷

نمایش سیگنال ۸

تحلیل فرکانسی ۱۱

تقسیم بندی سیگنال ۱۵

ادامه تقسیم بندی سیگنال ۱۶

استخراج ویژگی ۱۸

Iris Dataset ۱۹

بررسی اولیه ۱۹

تحلیل بصری داده ۲۵

گسسته سازی پیوستگی ۳۳

تحلیل آماری ۳۳

فهرست لینک های Google colab

1.jabr khati:
https://colab.research.google.com/drive/1HGU7_ze8LphAbqbCeD-PJas4Omcy-oN3?usp=sharing

https://colab.research.google.com/drive/1XSs3AlFpXPlNMUulN0B67BEDLoQiFlIF?usp=sharing

2. CWRU Dataset:

https://colab.research.google.com/drive/1JJJG7spy6UFujv1zuMm8C-Bgz1JywH5m?usp=sharing

3. Iris dataset:
https://colab.research.google.com/drive/1XSs3AlFpXPlNMUulN0B67BEDLoQiFlIF?usp=sharing

مقداری جبر خطی

الف)

وقتی دو ماتریس را ضرب می‌کنیم تعداد ستون‌های ماتریس اول باید با تعداد سطرهای ماتریس دوم برابر باشد و ماتریسی که در نهایت به دست می‌آید دارای تعداد سطرهای به اندازه ماتریس اول و ستون‌هایی به تعداد ستون‌های ماتریس دوم خواهد داشت با همین روش اندازه ضرب ماتریس‌ها را بدون محاسبه ماتریس‌ها بدست می‌آوریم; در ترانهاده تعداد سطرها و ستون‌های هر ماتریس جابجا می‌شود.

1. $\mathbf{BA} \Rightarrow \mathbf{B}_{4*2}, \mathbf{A}_{2*3} \Rightarrow \mathbf{BA}_{4*3}$
2. $\mathbf{B}_{4*2} \Rightarrow \mathbf{B}^{T}_{2*4}$
3. $\mathbf{B}^{T}\mathbf{A} \Rightarrow \mathbf{B}^{T}_{2*4}, \mathbf{A}_{2*3} \Rightarrow$ ضرب این دو ماتریس نیز تعریف شده نیست
۴. $\mathbf{A}^{T}\mathbf{B} \Rightarrow \mathbf{A}^{T}_{3*2}, \mathbf{B}_{4*2} \Rightarrow$ ضرب این دو ماتریس نیز تعریف شده نیست

ب)

$$\mathbf{A} \quad \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

$$\mathbf{B} \quad \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \\ b_{41} & b_{42} \end{bmatrix}$$

$$\mathbf{BA} \quad \begin{bmatrix} a_{11}b_{11} & a_{21}b_{12} & a_{12}b_{11} & a_{22}b_{12} & a_{13}b_{11} & a_{23}b_{12} \\ a_{11}b_{21} & a_{21}b_{22} & a_{12}b_{21} & a_{22}b_{22} & a_{13}b_{21} & a_{23}b_{22} \\ a_{11}b_{31} & a_{21}b_{32} & a_{12}b_{31} & a_{22}b_{32} & a_{13}b_{31} & a_{23}b_{32} \\ a_{11}b_{41} & a_{21}b_{42} & a_{12}b_{41} & a_{22}b_{42} & a_{13}b_{41} & a_{23}b_{42} \end{bmatrix}$$

$$\mathbf{B}^{\mathrm{T}} = \begin{bmatrix} b_{11} & b_{21} & b_{31} & b_{41} \\ b_{12} & b_{22} & b_{32} & b_{42} \end{bmatrix}$$

برای مورد ۳ و ۴ که تعریف شده نیستند، یک تابع در پایتون وجود دارد که اندازه سطر و ستون‌های ماتریس رو طوری تغییر می‌دهد که برای ضرب ماتریس‌ها مناسب باشد، به همین منظور از این تابع استفاده می‌کنیم

$\mathbf{B}^{T}\mathbf{A} \Rightarrow \mathbf{B}^{T}_{2*4}, \mathbf{A}_{2*3} \Rightarrow \mathbf{RB}^{T}_{4*2}, \mathbf{A}_{2*3}, \Rightarrow \mathbf{ARB}^{T}_{4*3}$

$$\mathbf{RB^T}=\begin{bmatrix} b_{11} & b_{21} \\ b_{31} & b_{41} \\ b_{12} & b_{22} \\ b_{32} & b_{42} \end{bmatrix}$$

که وقتی در ماتریس A ضرب شود ماتریس زیر نتیجه می‌شود:

$$\mathbf{RB^TA}=\begin{bmatrix} a_{11}b_{11}+a_{21}b_{21} & a_{12}b_{11}+a_{22}b_{21} & a_{13}b_{11}+a_{23}b_{21} \\ a_{11}b_{31}+a_{21}b_{41} & a_{12}b_{31}+a_{22}b_{41} & a_{13}b_{21}+a_{23}b_{41} \\ a_{11}b_{12}+a_{21}b_{22} & a_{12}b_{12}+a_{22}b_{22} & a_{13}b_{31}+a_{23}b_{22} \\ a_{11}b_{32}+a_{21}b_{42} & a_{12}b_{32}+a_{22}b_{42} & a_{13}b_{41}+a_{23}b_{42} \end{bmatrix}$$

کد این قسمت با استفاده از numpy:

```
import numpy as np

B=np.array([['b11','b12'],['b21','b22'],['b31','b32'],['b41','b42']])

A=np.array([['a11','a12','a13'],['a21','a22','a23']])

BT=B.T

BT.shape

RBT=np.reshape(BT,(4,2))

print(RBT)

B1=np.array([[1,2],[3,1],[4,5],[6,7]])

A1=np.array([[1,2,3],[4,5,6]])

BT1=B1.T

BT1.shape

RBT1=np.reshape(BT1,(4,2))

print(np.dot(RBT1,A1))
```

برای اینکه دو رشـــته را نمی‌توان در هم ضـــرب کرد مثال عددی به هر دو ماتریس می‌دهیم تا بتوانیم خروجی بگیریم.

خروجی‌ها به صورت زیر خواهد بود:

```
[['b11' 'b21']
 ['b31' 'b41']
 ['b12' 'b22']
 ['b32' 'b42']]
```

و

```
[[13 17 21]
 [28 38 48]
 [ 6  9 12]
 [33 45 57]]
```

A^TB => A^T_{3*2}, B_{4*2} => A^T_{3*2}, RB_{2*4} => A^TRB_{3*4}

$$\mathbf{A}^T = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}$$

$$\mathbf{RB} = \begin{bmatrix} b_{11} & b_{12} & b_{21} & b_{22} \\ b_{31} & b_{32} & b_{41} & b_{42} \end{bmatrix}$$

$$\mathbf{A^TRB} = \begin{bmatrix} a_{11}b_{11} + a_{21}b_{31} & a_{11}b_{12} + a_{21}b_{32} & a_{11}b_{21} + a_{21}b_{41} & a_{11}b_{22} + a_{21}b_{42} \\ a_{12}b_{11} + a_{22}b_{31} & a_{12}b_{12} + a_{22}b_{32} & a_{12}b_{21} + a_{22}b_{41} & a_{12}b_{22} + a_{22}b_{42} \\ a_{13}b_{11} + a_{21}b_{31} & a_{13}b_{12} + a_{21}b_{32} & a_{13}b_{21} + a_{21}b_{41} & a_{13}b_{22} + a_{21}b_{44} \end{bmatrix}$$

کد این قسمت با استفاده از numpy:

```
import numpy as np

B=np.array([['b11','b12'],['b21','b22'],['b31','b32'],['b41','b42']])

A=np.array([['a11','a12','a13'],['a21','a22','a23']])

B.shape

RB=np.reshape(B,(2,4))

AT=A.T
```

```
print(RB)

B1=np.array([[1,2],[3,1],[4,5],[6,7]])

A1=np.array([[1,2,3],[4,5,6]])

B1.shape

RB1=np.reshape(B1,(2,4))

AT1=A1.T

print(np.dot(AT1,RB1))
```

برای اینکه دو رشـــته را نمی‌توان در هم ضـــرب کرد مثال عددی به هر دو ماتریس می‌دهیم تا بتوانیم خروجی بگیریم.

خروجی‌ها به صورت زیر خواهد بود:

```
[['b11' 'b12' 'b21' 'b22']
 ['b31' 'b32' 'b41' 'b42']]
```

و

```
[[17 22 27 29]
 [22 29 36 37]
 [27 36 45 45]]
```

سوال دو

$$\mathbf{x}^{(\mathrm{i})} = \begin{bmatrix} \mathrm{x}_1^{(\mathrm{i})} & \mathrm{x}_2^{(\mathrm{i})} \end{bmatrix}, \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

$$\mathbf{x}^{\boldsymbol{i}}\boldsymbol{\theta} = \left[\mathrm{x}_1^{(\mathrm{i})}\theta_1 + \mathrm{x}_2^{(\mathrm{i})}\theta_2 \right]_{1*1}$$

$$\mathbf{X\theta} = \begin{bmatrix} \mathrm{x}_{11}^{(\mathrm{i})}\theta_1 + \mathrm{x}_{12}^{(\mathrm{i})}\theta_2 \\ \vdots \\ \mathrm{x}_{\mathrm{n1}}^{(\mathrm{i})}\theta_1 + \mathrm{x}_{\mathrm{n2}}^{(\mathrm{i})}\theta_2 \end{bmatrix}_{n*1}$$

$$\mathbf{J} = \sum_{i=1}^{n} (\mathbf{x}^{(\mathbf{i})}\boldsymbol{\theta} - \mathbf{y}^{(i)})^2 \rightarrow J = (\mathbf{X\theta} - \mathbf{Y})^T (\mathbf{X\theta} - \mathbf{Y})$$

$$(\mathbf{X\theta} - \mathbf{Y}) = \begin{bmatrix} \mathrm{x}_{11}^{(1)}\theta_1 + \mathrm{x}_{12}^{(1)}\theta_2 - y^{(1)} \\ \vdots \\ \mathrm{x}_{\mathrm{n1}}^{(\mathrm{n})}\theta_1 + \mathrm{x}_{\mathrm{n2}}^{(\mathrm{n})}\theta_2 - y^{(n)} \end{bmatrix}_{n*1}$$

$$(\mathbf{X\theta}-\mathbf{Y})^T=\left[x_{11}^{(1)}\theta_1+x_{12}^{(1)}\theta_2-y^{(1)}\quad\cdots\quad x_{n1}^{(n)}\theta_1+x_{n2}^{(n)}\theta_2-y^{(n)}\right]_{1*n}$$

$$\mathbf{J}=\left[(x_{11}^{(1)}\theta_1+x_{12}^{(1)}\theta_2-y^{(1)})^2+(x_{21}^{(2)}\theta_1+x_{22}^{(2)}\theta_2-y^{(2)})^2+\ldots+(x_{n1}^{(n)}\theta_1+x_{n2}^{(n)}\theta_2-y^{(n)})^2\right]_{1*1}$$

$$\nabla_{\theta}\mathbf{J}=\begin{bmatrix}\frac{\partial(x_{11}^{(1)}\theta_1+x_{12}^{(1)}\theta_2-y^{(1)})^2+(x_{21}^{(2)}\theta_1+x_{22}^{(2)}\theta_2-y^{(2)})^2+\ldots+(x_{n1}^{(n)}\theta_1+x_{n2}^{(n)}\theta_2-y^{(n)})^2}{\partial\theta_1}\\ \frac{\partial(x_{11}^{(1)}\theta_1+x_{12}^{(1)}\theta_2-y^{(1)})^2+(x_{21}^{(2)}\theta_1+x_{22}^{(2)}\theta_2-y^{(2)})^2+\ldots+(x_{n1}^{(n)}\theta_1+x_{n2}^{(n)}\theta_2-y^{(n)})^2}{\partial\theta_2}\end{bmatrix}_{2*1}$$

پردازش داده

CWRU Dataset

دریافت داده

۱. فرمت فایل دانلودmat. بوده که درواقع برای متلب هست و برای خواندن آن باید از کتابخانه scipy.io باید loadmat را فراخوانی کرده و با استفاده از آن فایل را بخوانیم، مانند زیر:

```
•[69]: from scipy.io import loadmat
       data=loadmat(r'201.mat')
```

در واقع با این کد کتابخانه فراخوانی شده و سپس یک متغیر را به دیتای خوانده شده نسبت می دهیم، از این پس می توان با فراخوانی متغیر data در کد، به این دیتا دسترسی داشت.

۲. در این بخش برای بدست اوردن مشخصات داده از type آن باید اطلاعات کسب کنیم که نتیجه نشان می دهد که دیکشنری هست. برای اطلاعات دیگر باید از تابع data.keys() استفاده کنیم که در واقع برای نشان دادن کلید ها یا اجزای داده به کار میروند و موارد زیر را شامل می شود:

Header

,version

, global

که در واقع هر کدام از اینها اشاره دارند به:

۱. اطلاعات کلی درباره فایلmat

۲.نسخه فایل mat

۳. متغیرهای کلی ذخیره شده در این فایل

درواقع X201_DE_time شامل سیگنال های اصلی ذخیره شده می باشد.

کد :

```
from scipy.io import loadmat
data=loadmat(r'201.mat')
print("نوع داده خوانده شده :", type(data))

print('keys:',data.keys())
```

خروجی:

```
نوع داده خوانده شده : <class 'dict'>
keys: dict_keys(['__header__', '__version__', '__globals__', 'X201_DE_time', 'X201_FE_time', 'X201RPM'])
```

نمایش سیگنال

کد:

```
signal = data['X201_DE_time'][:2000:2]
fs = 48000

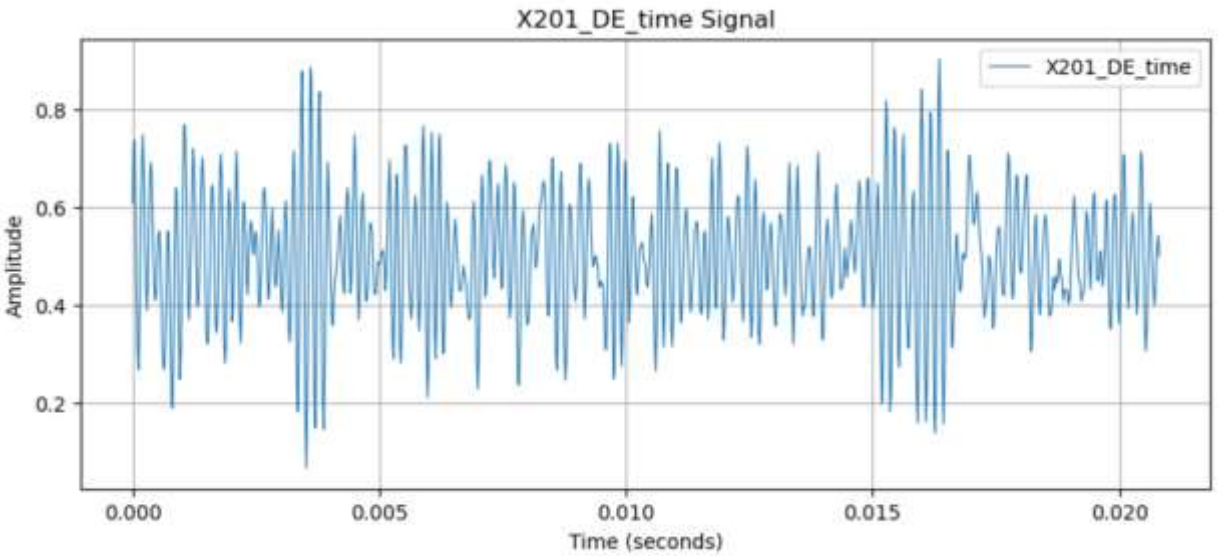
#calculating time vector
time = np.arange(len(signal)) / fs

plt.figure(figsize=(10, 4))
plt.plot(time, signal, label="X201_DE_time", linewidth=0.8)
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.title("X201_DE_time Signal")
plt.legend()
plt.grid()
plt.show()
```

در این قسمت باید از بین سیگنال های موجود به دلخواه سیگنالی را انتخاب می کردیم، سیگنال انتخابی ما بدین صورت تعریف شده است که از میان دیتا های ۰ تا ۲۰۰۰ با فاصله های دوتایی سیگنال را انتخاب کنیم (خط ابتدایی کد) سپس باید برای محور افقی زمان را تعریف کنیم برای اینکار باید نرخ بهره برداری را لحاظ کنیم که ۴۸۰۰۰ در نظر گرفته شده است سپس با استفاده از تابع arrange زمان را محاسبه کرده و در متغیری ذخیره می کنیم. برای رسم نمودار طبق مطالعات انجام شده می توانیم از دستوراتی که آورده شده استفاده کنیم که ابتدا اندازه فیگور را مشخص کرده و سپس نوع نمودار را مشخص می کنیم و پس از آن

x,y و ضخامت نمودار ترسیمی مشخص می شود و از لیبل ها برای جلوه بصری نمودار و مشخص سازی نمودار های افقی و عمودی استفاده کرده ایم، اینجا زیاد نیازی به راهنمای نمودار نیست چون تنها یک نمودار رسم شده با این حال راهنما هم درج شده تا کاملا مشخص باشد.

خروجی:



۲. حال باید سیگنال را در بازه ی زمانی مشخصی نشان دهیم، در این قسمت برای آنکه نوعی سیگنال دیگر را هم بتوانیم در نظر بگیریم سیگنال انتخابی را تغییر داده و دوباره در متغیری ذخیره میکنیم

کد:

```
signal = data['X201_DE_time'].flatten()

fs = 48000

# Calculating time vector
time = np.arange(len(signal)) / fs

# Define the time range for 2s to 2.01s
start_time = 2.0
end_time = 2.01

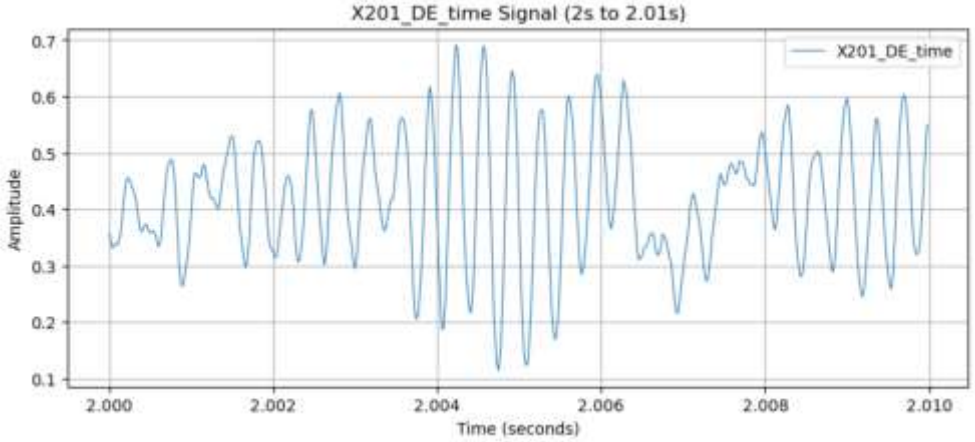
# Find the indices corresponding to the selected time range
start_index = np.searchsorted(time, start_time)
end_index = np.searchsorted(time, end_time)

# Extract the corresponding time and signal values
time_selected = time[start_index:end_index]
signal_selected = signal[start_index:end_index]

# Plot the selected time range of the signal
plt.figure(figsize=(10, 4))
plt.plot(time_selected, signal_selected, label="X201_DE_time", linewidth=0.8)
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.title("X201_DE_time Signal (2s to 2.01s)")
plt.legend()
plt.grid()
plt.show()
```

منطق این کد به این صورت است که باید محور افقی را جوری اصلاح کنیم که تنها اعداد در بازه ۲ثانیه تا ۲.۰۱ ثانیه را داشته باشد پس باید ابتدا دو متغیر به این دو عدد نسبت بدهیم و سپس از میان گستره زمانی مرتب شده که در متغیری ذخیره شده است شروع به جستجوی اعداد در این بازه کنیم در اینجا از یک متد جدید به عنوان searchsorted استفاده شده که این کار جستجوی مرتب شده را برای ما انجام می دهد. سپس مانند قسمت پیش به رسم نمودار می پردازیم، باز هم نیاز به راهنما نبوده اما برای تکمیل بودن نمودار آنرا اضافه کردیم و صفحه را شطرنجی کرده تا تحلیل داده راحت تر امکان پذیر باشد.

خروجی:



تحلیل فرکانسی

۱. کد :

```python
import numpy as np
import matplotlib.pyplot as plt

def compute_fft(signal, sampling_rate):
    N = len(signal)  # تعداد نمونه‌ها
    freq = np.fft.fftfreq(N, d=1/sampling_rate)
    fft_values = np.fft.fft(signal)

    plt.figure(figsize=(8, 4))
    plt.plot(freq[:N//2], np.abs(fft_values[:N//2]))
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Amplitude')
    plt.title('Fourier Transform of Signal')
    plt.grid()
    plt.show()
```

ابتدا یک تابع برای تبدیل سیگنال در حوزه زمان و تبدیل فوریه می نویسیم، در این تابع ورودی ها نرخ نمونه برداری و خود سیگنال هستند، سپس تعداد کل نمونه ها را حساب می کنیم، پس از آن:

np.fft.fftfreq(N, d=1/sampling_rate) فرکانس‌های متناظر با هر مقدار در تبدیل فوریه را محاسبه می‌کند .

d=1/sampling_rate فاصله‌ی بین نمونه‌ها را مشخص می‌کند.

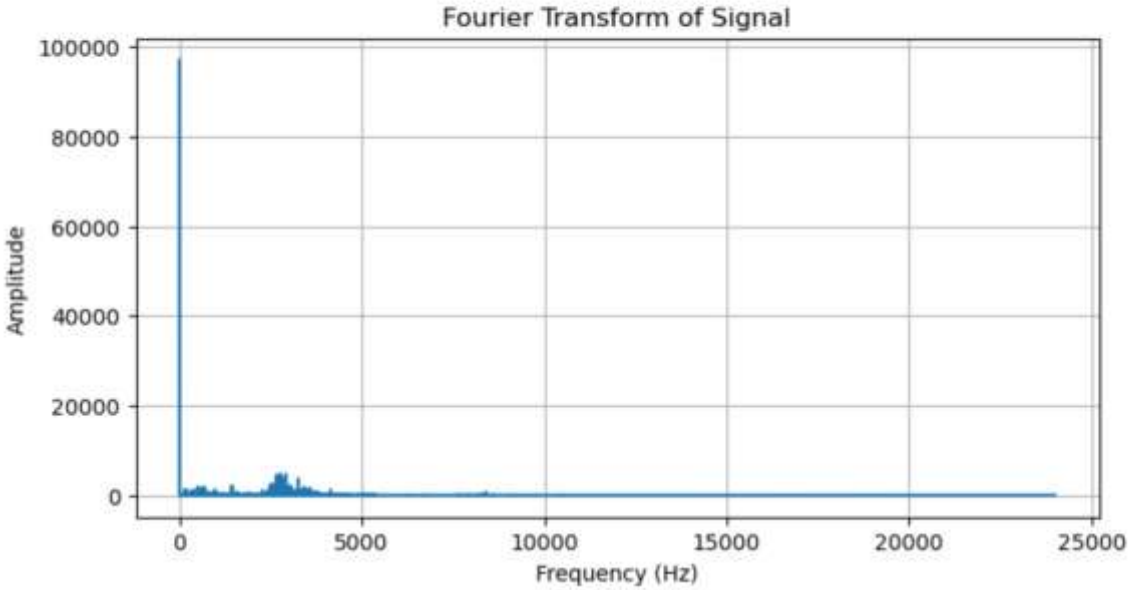
پس از آن تابع np.fft.fft(signal) تبدیل فوریه‌ی سریع (FFT) را محاسبه می‌کند و مقدار آن در fft_values ذخیره می‌شود. حال باید نمودار ترسیم شود، به تکرار مکررات نمی پردازیم و صرفا بیان می کنیم که

np.abs(fft_values) مقدار دامنه (قدر مطلق) را محاسبه می‌کند .

فقط نیمه‌ی مثبت ([:N//2]) رسم می‌شود، زیرا خروجی FFT متقارن است.

نحوه استفاده از تابع تعریف شده و رسم سیگنال اخر:

```
fs = 48000
signal = data['X201_DE_time'].flatten()
compute_fft(signal, fs)
```



حال برای قسمت دوم که فرکانس غالب از ما سوال شده است، به این صورت عمل می کنیم:

ابتدا تابع مورد نظر را ادیت کرده و بهتر و واضح تر متغیرها را تغریف می کنیم و سپس مشاهده می کنیم که فرکانس غالبب ۰ شده است که این به دلایل مختلفی اتفاق می افتد، یکی از راهکار ها حذف کردن فرکانس میانگین است . پس ابتدا کد و خروجی حالت اول را مشاهده می کنیم و سپس حالت دوم که فرکانس غالب به ۰.۱۹۶ هرتز رسیده است:

کد۱:

```
signal = data['X201_DE_time'].flatten()
fs = 48000

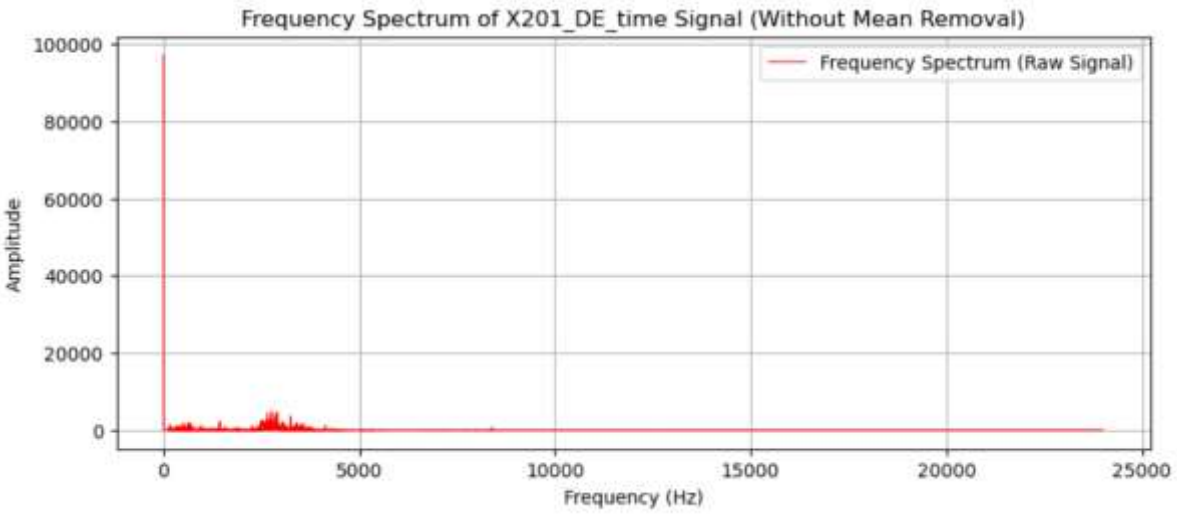
def plot_frequency_spectrum_raw(signal, fs):
    N = len(signal)

    fft_values = np.fft.fft(signal)

    freq = np.fft.fftfreq(N, d=1/fs)

    positive_freqs = freq[:N//2]
    positive_fft_values = np.abs(fft_values[:N//2])
        plt.figure(figsize=(10, 4))
    plt.plot(positive_freqs, positive_fft_values, label="Frequency Spectrum (Raw Signal)", linewidth=0.8, color='r')
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Amplitude")
    plt.title("Frequency Spectrum of X201_DE_time Signal (Without Mean Removal)")
    plt.legend()
    plt.grid(True)
    plt.show()
plot_frequency_spectrum_raw(signal, fs)
```

خروجی۱:



کد۲:

منطق کد دوم به این صورت است :

محاسبه و نمایش طیف فرکانسی سیگنال با استفاده از تبدیل فوریه(FFT)

یافتن فرکانس غالب پس از حذف مقدار میانگین سیگنال

بنابراین:

حذف مقدار میانگین سیگنال

- این کار باعث می‌شود مؤلفه DC حذف شود و تحلیل طیف فرکانسی دقیق‌تر شود.

محاسبه و نمایش طیف فرکانسی با استفاده از FFT

- تابع plot_frequency_spectrum()تبدیل فوریه را انجام داده و طیف را نمایش می‌دهد.

یافتن فرکانس غالب

- تابع find_dominant_frequency()از بین فرکانس‌های موجود، بیشترین دامنه را پیدا کرده و فرکانس غالب را برمی‌گرداند.

و در نهایت هم :

ابتدا طیف فرکانسی سیگنال نمایش داده می‌شود .

سپس فرکانس غالب محاسبه و در خروجی چاپ می‌شود

کد:

```
signal_detrended = signal - np.mean(signal)

def plot_frequency_spectrum(signal, fs):

    N = len(signal)
    fft_values = np.fft.fft(signal)
    freq = np.fft.fftfreq(N, d=1/fs)
    positive_freqs = freq[:N//2]
    positive_fft_values = np.abs(fft_values[:N//2])

    plt.figure(figsize=(10, 4))
    plt.plot(positive_freqs, positive_fft_values, label="Frequency Spectrum", linewidth=0.8)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Amplitude")
    plt.title("Frequency Spectrum of X201_DE_time Signal (After Mean Removal)")
    plt.legend()
    plt.grid()
    plt.show()

def find_dominant_frequency(signal, fs):
    fft_values = np.fft.fft(signal)
    freq = np.fft.fftfreq(len(signal), d=1/fs)

    positive_freqs = freq[:len(signal)//2]
    positive_fft_values = np.abs(fft_values[:len(signal)//2])

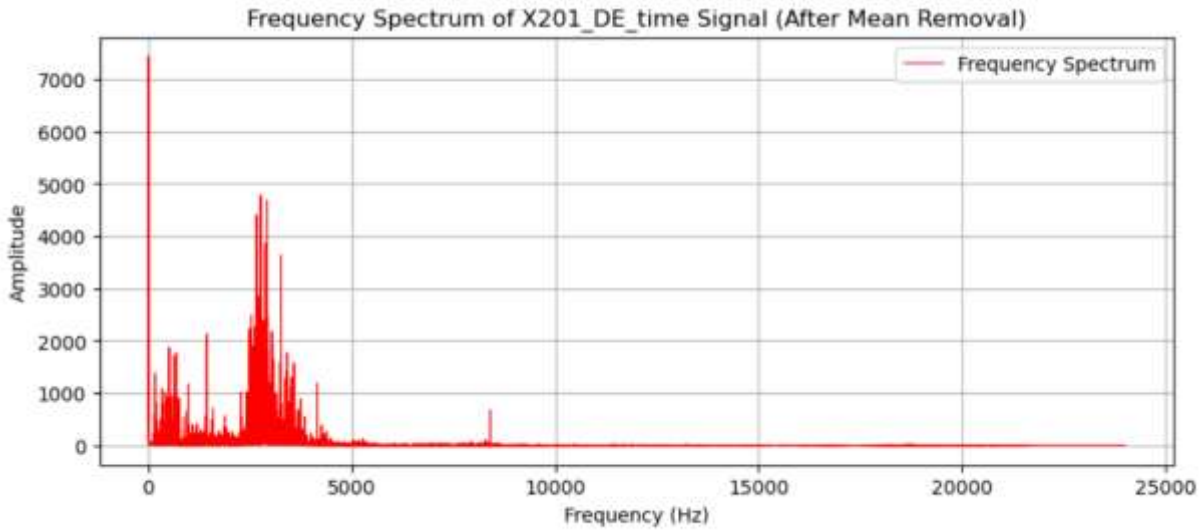
    dominant_frequency = positive_freqs[np.argmax(positive_fft_values)]

    return dominant_frequency

plot_frequency_spectrum(signal_detrended, fs)


dominant_freq_corrected = find_dominant_frequency(signal_detrended, fs)
print(f"فرکانس غالب سیگنال پس از حذف مقدار میانگین: {dominant_freq_corrected:.3f} Hz")
```

خروجی۲:



فرکانس غالب سیگنال پس از حذف مقدار میانگین: 0.196 Hz

تقسیم بندی سیگنال

کد:

```
signal = data['X201_DE_time'].flatten()
signal_detrended = signal - np.mean(signal)
segment_length = 128
overlap = 64

def segment_signal(signal, segment_length=128, overlap=64):

    step = segment_length - overlap
    segments = []

    for start in range(0, len(signal) - segment_length + 1, step):
        segment = signal[start:start + segment_length]
        segments.append(segment)

    return np.array(segments)

segmented_signal = segment_signal(signal_detrended, segment_length, overlap)

df_segments = pd.DataFrame(segmented_signal)
```

در اینجا میزان همپوشانی و سیگنال بدون وجود میانگین آنرا تعریف می کنیم. طول تقسیم بندی ۱۲۸ درنظر گرفته شده است (خط اول تا سوم initialization است)

سپس به تقسیم بندی با تابع می پردازیم در تابع ابتدا ورودی ها که خود سیگنال و میزان همپوشانی و طول تقسیم است را مشخص کرده سپس از یک حلقه برای درست کردن آرایه ی قطعات ترتیب می دهیم. حال در حلقه forداریم:

منطق: یک سیگنال را به قطعه‌های (segment) کوچکتر با طول segment_lengthتقسیم می‌کند و این قطعه‌ها را در یک آرایه numpyذخیره می‌کند. هر قطعه می‌تواند مقداری **همپوشانی (overlap)** با قطعه‌های قبلی داشته باشد که مقدار این همپوشانی با stepتنظیم می‌شود. سپس ارایه را باز میگرداند در حلقه اصلی برنامه تا از آن استفاده کنیم.

در قسمت پایانی کد تابع را روی سیگنال مدنظر اعمال کرده و سپس با استفاده از دستور DataFrame آنرا تبدیل کرده و در نهایت خروجی می گیریم.

خروجی:

	0	1	2	3	4	5	6	7	8	9	...	118	119	120	121	
0	0.213103	0.279860	0.334935	0.359551	0.340150	0.280695	0.204133	0.116514	0.031608	-0.038070	...	0.322835	0.319914	0.286745	0.246065	0.199
1	-0.061852	-0.004900	0.059145	0.115263	0.154482	0.169503	0.153648	0.119435	0.060188	-0.015122	...	-0.080627	-0.018877	0.063526	0.139462	0.201
2	0.003236	0.030773	0.065612	0.110465	0.169294	0.221865	0.277148	0.305937	0.304894	0.289457	...	0.000107	-0.009698	0.005948	0.033485	0.063
3	0.086474	0.018257	-0.029725	-0.031185	-0.001145	0.051009	0.114220	0.173884	0.229793	0.282364	...	0.106292	0.194328	0.262962	0.306354	0.318
4	0.243770	0.236885	0.225412	0.202255	0.171172	0.122356	0.075834	0.037449	0.016170	0.021594	...	-0.122767	-0.027013	0.072705	0.153857	0.224
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
3824	-0.153643	-0.122142	-0.100237	-0.073951	-0.045163	-0.009907	0.015545	0.017631	0.001567	-0.041407	...	-0.113797	-0.115883	-0.111502	-0.092518	-0.090
3825	-0.024927	0.010329	0.022220	0.031399	0.020134	-0.015748	-0.065607	-0.148844	-0.209760	-0.249188	...	-0.054550	-0.037235	-0.014705	0.002610	0.001
3826	-0.062269	-0.084382	-0.100863	-0.115883	-0.133407	-0.159692	-0.193697	-0.210803	-0.213515	-0.195783	...	-0.039947	-0.002188	0.016379	0.023055	0.031
3827	-0.140708	-0.185978	-0.212263	-0.212889	-0.204336	-0.171375	-0.134241	-0.087720	-0.032437	0.003862	...	-0.050795	-0.064147	-0.102949	-0.122559	-0.141
3828	-0.072908	-0.073951	-0.071448	-0.079793	-0.090432	-0.092101	-0.072491	-0.040782	-0.012619	0.009286	...	-0.144672	-0.087303	-0.031185	0.019925	0.049

3829 rows × 128 columns

ادامه تقسیم بندی سیگنال

کد:

```
selected_indices = [i for i in range(0, len(df_segments), 13)][:10]
colors = plt.cm.viridis(np.linspace(0, 1, len(selected_indices)))
plt.figure(figsize=(10, 5))

for i, (idx, color) in enumerate(zip(selected_indices, colors)):
    plt.plot(df_segments.iloc[idx], label=f"Segment {idx}", color=color)

plt.xlabel("Sample Index")
plt.ylabel("Amplitude")
plt.title("Selected Segments from Signal")
plt.legend()
plt.grid()
plt.show()
```

توضیحات کد:

این خط لیستی از **۱۰ اندیس** را انتخاب می‌کند که نشان‌دهنده برخی قطعه‌های موجود در df_segmentsهستند.

range(0, len(df_segments), 13)یعنی از صفر شروع کن و هر ۱۳ قطعه یک‌بار یک قطعه انتخاب کن.

[:10]یعنی فقط **۱۰ قطعه‌ی اول** را بردار.

هدف :ما قرار نیست همه‌ی قطعه‌های سیگنال را نمایش دهیم، بلکه فقط **هر سیزدهمین قطعه تا حداکثر ۱۰ مورد** را انتخاب می‌کنیم تا نمودار شلوغ نشود.

در قسمت بعدی داریم:

np.linspace(0, 1, len(selected_indices))یک لیست از **عددهای یکنواخت بین ۰ تا ۱** ایجاد می‌کند که به ما امکان می‌دهد برای هر قطعه یک رنگ متفاوت در طیف رنگی **Viridis**انتخاب کنیم.

plt.cm.viridis(...)این عددهای بین ۰ تا ۱ را به رنگ‌های واقعی در طیف **Viridis**(زرد تا بنفش) تبدیل می‌کند.

هدف :هر قطعه سیگنال در نمودار با رنگی متفاوت نمایش داده شود تا قابل تشخیص باشد.

حلقه بعدی **هر قطعه‌ی انتخاب‌شده را روی نمودار رسم می‌کند.**

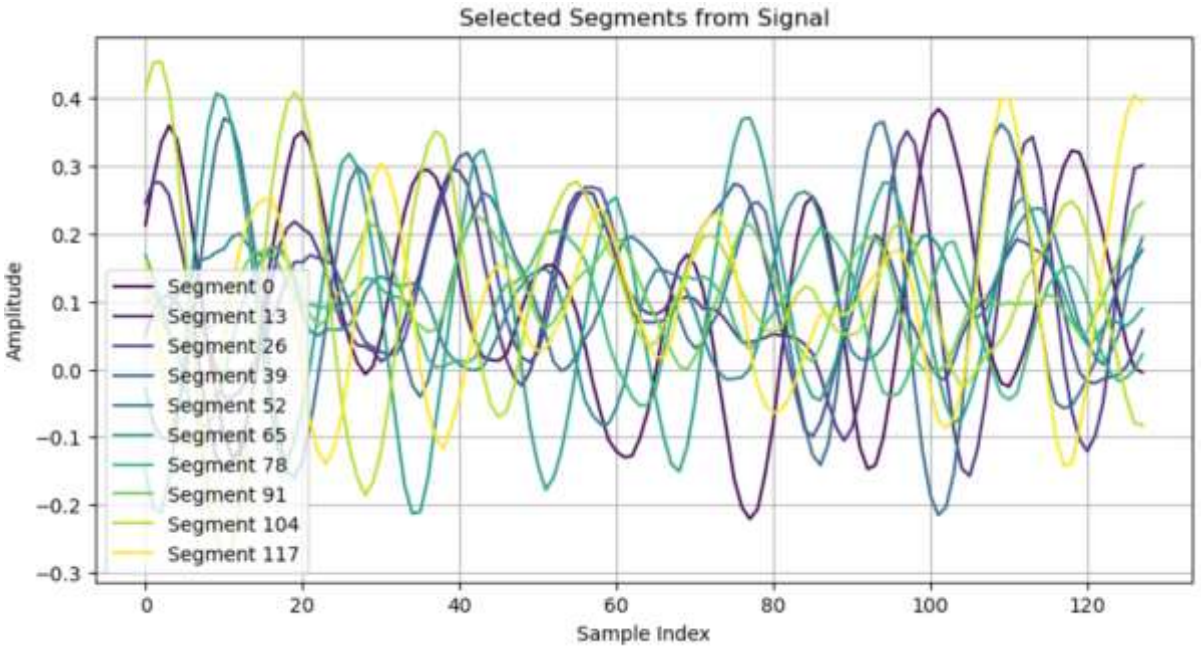
df_segments.iloc[idx]همان **قطعه‌ی idx ام** از سیگنال اصلی است.

label=f"Segment {idx}"نام قطعه را روی نمودار نشان می‌دهد.

color=colorباعث می‌شود هر قطعه رنگ مخصوص خودش را داشته باشد.

و در نهایت هم جزئیات رسم نمودار است که در بخش های قبلی توضیح داده شده است.

خروجی:



استخراج ویژگی

کد:

```
def extract_features(signal):
    mean_value = np.mean(signal)
    std_value = np.std(signal)
    rms_value = np.sqrt(np.mean(np.square(signal)))
    return mean_value, std_value, rms_value

features = []
for i in range(len(df_segments)):
    mean_val, std_val, rms_val = extract_features(df_segments.iloc[i])
    features.append([mean_val, std_val, rms_val])

df_features = pd.DataFrame(features, columns=["Mean", "Std", "RMS"])

csv_filename = "Hoosh1.csv"
df_features.to_csv(csv_filename, index=False)

print(f"ویژگی‌های استخراج‌شده در فایل '{csv_filename}' ذخیره شد.")
```

```
ویژگی‌های استخراج‌شده در فایل 'Hoosh1.csv' ذخیره شد.
```

ابتدا یک تابع تعریف شده که سه ویژگی مهم از هر قطعه سیگنال را محاسبه می‌کند: میانگین، انحراف معیار و مقدار ریشه میانگین مربعات. میانگین نشان‌دهنده مقدار مرکزی سیگنال است، انحراف معیار میزان پراکندگی مقادیر سیگنال را مشخص می‌کند، و مقدار ریشه میانگین مربعات به عنوان یک شاخص کلی برای انرژی سیگنال عمل می‌کند.

سپس یک حلقه روی تمام قطعه‌های سیگنال اجرا می‌شود. در هر تکرار، ویژگی‌های آماری مربوط به یک قطعه محاسبه شده و در لیستی ذخیره می‌شود. این لیست شامل سه مقدار برای هر قطعه است که به‌ترتیب میانگین، انحراف معیار و مقدار ریشه میانگین مربعات آن را در بر دارد.

پس از تکمیل این پردازش، لیست ویژگی‌ها به یک دیتافریم تبدیل می‌شود که سه ستون مجزا برای هر ویژگی دارد. این کار باعث می‌شود که داده‌ها در یک قالب جدولی سازمان‌یافته شوند و به‌راحتی مورد تجزیه و تحلیل قرار گیرند.

در نهایت، دیتافریم ایجاد شده به صورت یک فایل CSV ذخیره می‌شود تا بتوان از آن در مراحل بعدی، مانند یادگیری ماشین یا تحلیل‌های بیشتر، استفاده کرد. همچنین یک پیام نمایش داده می‌شود تا کاربر بداند که عملیات ذخیره‌سازی با موفقیت انجام شده است.

Iris Dataset

بررسی اولیه

۱. تحقیق درباره این دیتاست

مجموعه داده **Iris**یکی از معروف‌ترین و پرکاربردترین مجموعه داده‌ها در دنیای یادگیری ماشـین و آمار اسـت. این مجموعه برای اولین بار توسط **رونالد فیشـر** (Ronald Fisher)در سال ۱۹۳۶ معرفی شد و هدف آن طبقه‌بندی گونه‌های مختلف گل زنبق (**Iris**)بر اساس ویژگی‌های ظاهری آن‌ها بود.

اجزای مجموعه داده Iris

این مجموعه شامل اطلاعات مربوط به **150نمونه گل زنبق** است که در سه گونه مختلف دسته‌بندی شده‌اند:

۱. **Iris Setosa**

۲. **Iris Versicolor**

۳. **Iris Virginica**

هر نمونه دارای **4ویژگی عددی** است که ابعاد مختلف گلبرگ و کاسبرگ را نشان می‌دهند:

- **طول کاسبرگ – (Sepal Length) بر حسب سانتی‌متر**
- **عرض کاسبرگ – (Sepal Width) بر حسب سانتی‌متر**
- **طول گلبرگ – (Petal Length) بر حسب سانتی‌متر**
- **عرض گلبرگ – (Petal Width) بر حسب سانتی‌متر**

همچنین، هر نمونه دارای **یک برچسب کلاس (Class Label)** است که مشخص می‌کند این نمونه به کدام گونه تعلق دارد.

چرا مجموعه داده Iris مهم است؟

- **سـاده ولی مفید** :این مجموعه داده از نظر ابعاد کوچک اسـت (فقط ۱۵۰ نمونه و ۴ ویژگی)، اما همچنان برای تمرین الگوریتم‌های طبقه‌بندی بسیار مفید است.
- **مناسـب برای الگوریتم‌های یادگیری ماشـین** :بسـیاری از مدل‌های یادگیری ماشـین، از جمله **درخت تصـمیم، رگرسـیون لجسـتیک، K-نزدیک‌ترین همسـایه (KNN) و شـبکه‌های عصـبی**، روی این مجموعه داده آزمایش شده‌اند.

- **مصورسازی آسان** :با توجه به تعداد کم ویژگی‌ها، امکان نمایش آن‌ها به‌صورت نمودارهای دو بعدی و سه بعدی وجود دارد که به درک بهتر داده‌ها کمک می‌کند.

کاربردهای مجموعه داده Iris

این مجموعه معمولاً برای آزمایش و یادگیری مفاهیم پایه‌ای یادگیری ماشین مانند:

طبقه‌بندی(**Classification**)

کاهش ابعاد(**Dimensionality Reduction**)

خوشه‌بندی(**Clustering**)

مصورسازی داده(**Data Visualization**)

قرار می‌گیرد.

نحوه خواندن و ایمپورت کردن دیتای ایریس:

اینجا دیتای آیریس را از دیتاست های sklearn لود می کنیم و در متغیری ذخیره میکنیم.

```
from sklearn import datasets
iris_data=datasets.load_iris()
```

۳و۴. تقسیم داده به دیتای train , test و تبدیل به دیتافریم

کد:

```
[3]: iris_data=datasets.load_iris()
```

```
[63]: X, y = iris_data.data, iris_data.target

#20%train , 80%test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("ابعاد مجموعه آموزش:", X_train.shape, y_train.shape)
print("ابعاد مجموعه آزمون:", X_test.shape, y_test.shape)
print('Xtrain:',X_train[:5:1])
print('ytrain:',y_train[:5:1])
print('Xtest:',X_test[:5:1])
print('ytest:',y_test[:5:1])
```

در این بخش دیتای مدنظر خوانده شده و سپس با استفاده از متد split آنرا به دو دسته آزمون و آموزش تقسیم کرده ایم و برای اطمینان خاطر ابعاد آنها را نمایش داده و ۵ تای اول هر کدام را پرینت کرده ایم.

خروجی:

```
ابعاد مجموعه آموزش: (120, 4) (,120)
ابعاد مجموعه آزمون: (30, 4) (,30)
Xtrain: [[4.6 3.6 1.  0.2]
 [5.7 4.4 1.5 0.4]
 [6.7 3.1 4.4 1.4]
 [4.8 3.4 1.6 0.2]
 [4.4 3.2 1.3 0.2]]
ytrain: [0 0 1 0 0]
Xtest: [[6.1 2.8 4.7 1.2]
 [5.7 3.8 1.7 0.3]
 [7.7 2.6 6.9 2.3]
 [6.  2.9 4.5 1.5]
 [6.8 2.8 4.8 1.4]]
ytest: [1 0 2 1 1]
```

در قسمت بعد باید این دو دسته را تبدیل به دیتا فریم کنیم، کاری که اینجا صورت گرفته است به این منوال است که ابتدا ایکس و ایگرگ های ترین و تست تبدیل به دیتا فریم شده اند و سپس به هم اضافه شده ند تا در نهایت دو دیتا فریم تست و ترین داشته باشیم کد های مربوط به این صورت هستند:

```
column_names = iris_data.feature_names
X_test_df = pd.DataFrame(X_test, columns=column_names)
y_test_df = pd.DataFrame(y_test, columns=["target"])
print(X_test_df)
print(y_test_df)
```

خروجی:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	6.1	2.8	4.7	1.2
1	5.7	3.8	1.7	0.3
2	7.7	2.6	6.9	2.3
3	6.0	2.9	4.5	1.5
4	6.8	2.8	4.8	1.4
5	5.4	3.4	1.5	0.4
6	5.6	2.9	3.6	1.3
7	6.9	3.1	5.1	2.3
8	6.2	2.2	4.5	1.5
9	5.8	2.7	3.9	1.2
10	6.5	3.2	5.1	2.0
11	4.8	3.0	1.4	0.1
12	5.5	3.5	1.3	0.2
13	4.9	3.1	1.5	0.1
14	5.1	3.8	1.5	0.3
15	6.3	3.3	4.7	1.6
16	6.5	3.0	5.8	2.2
17	5.6	2.5	3.9	1.1
18	5.7	2.8	4.5	1.3
19	6.4	2.8	5.6	2.2
20	4.7	3.2	1.6	0.2
21	6.1	3.0	4.9	1.8
22	5.0	3.4	1.6	0.4
23	6.4	2.8	5.6	2.1
24	7.9	3.8	6.4	2.0
25	6.7	3.0	5.2	2.3
26	6.7	2.5	5.8	1.8
27	6.8	3.2	5.9	2.3
28	4.8	3.0	1.4	0.3
29	4.8	3.1	1.6	0.2

	target
0	1
1	0
2	2
3	1
4	1
5	0
6	1
7	2
8	1
9	1
10	2
11	0
12	0
13	0
14	0
15	1
16	2
17	1
18	1
19	2
20	0
21	2
22	0
23	2
24	2
25	2
26	2
27	2
28	0

کد بخشی که ستون ایگرگ به قبلی ها اضافه می شوند:

```
train_df = pd.DataFrame(X_train, columns=column_names)
train_df['target'] = y_train

test_df = pd.DataFrame(X_test, columns=column_names)
test_df['target'] = y_test

test_df.head()
```

خروجی ها:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	6.1	2.8	4.7	1.2	1
1	5.7	3.8	1.7	0.3	0
2	7.7	2.6	6.9	2.3	2
3	6.0	2.9	4.5	1.5	1
4	6.8	2.8	4.8	1.4	1

```
train_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	4.6	3.6	1.0	0.2	0
1	5.7	4.4	1.5	0.4	0
2	6.7	3.1	4.4	1.4	1
3	4.8	3.4	1.6	0.2	0
4	4.4	3.2	1.3	0.2	0

۵. در این بخش باید دو دیتا فریم را ادغام کنیم و نشـان دهیم که هر دیتا مربوط به دیتافریم test اسـت یا دیتافریم train برای اینکار ابتدا به دیتافریم های هریک یک ستون اضافه می کنیم و دسته بندی را مشخص می کنیم(۲ خط اول) سپس با استفاده از دستور concat دو دیتافریم را با یکدیگر ترکیب می کنیم و با فراخوانی اسم آن در خروجی نمایش می دهیم.

کد:

```
train_df['class'] = 'train'
test_df['class'] = 'test'

total_df = pd.concat([train_df, test_df], ignore_index=True)

total_df
```

خروجی: ستون اخر دسته بندی را نشان می دهد به علاوه از ignore_index=True استفاده شده تا شمارش ایندکس ها به ترتیب باشد و ایندکس های اصلی دیتافریم ها نادیده گرفته شوند.

[71]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	class
0	4.6	3.6	1.0	0.2	0	train
1	5.7	4.4	1.5	0.4	0	train
2	6.7	3.1	4.4	1.4	1	train
3	4.8	3.4	1.6	0.2	0	train
4	4.4	3.2	1.3	0.2	0	train
...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2	test
146	6.7	2.5	5.8	1.8	2	test
147	6.8	3.2	5.9	2.3	2	test
148	4.8	3.0	1.4	0.3	0	test
149	4.8	3.1	1.6	0.2	0	test

تحلیل بصری داده

۱. دوبعدی: برای این رسم از کتابخانه seaborn که ویژگی های تخصصی تری را در اختیار ما قرار می دهد استفاده می کنیم.
کتابخانه های اعمال شده به این صورت هستند:

import matplotlib.pyplot as plt

import seaborn as sns

در ادامه کد را بارگذاری کرده و درباره خط به خط آن توضیح می دهیم:

```
sns.set_style("darkgrid")
sns.set_context("talk")

feature_x = "petal length (cm)"
feature_y = "petal width (cm)"


plt.figure(figsize=(9, 6))


scatter = sns.scatterplot(
    data=total_df,
    x=feature_x,
    y=feature_y,
    hue="target",
    style="class",
    palette="deep",
    s=120,
    edgecolor="black",
    alpha=0.5 )

plt.xlabel("petal length (cm)", fontsize=14, fontweight="bold")
plt.ylabel("petal width (cm)", fontsize=14, fontweight="bold")
plt.title("Dispersion ", fontsize=16, fontweight="bold", color="darkblue")


plt.legend(title="type ", title_fontsize="13", fontsize="11", loc="upper left")
plt.show()
```

در اینجا ابتدا style نمودار را مشخص می کنیم و درخط بعدی فونت را مشخص میکنیم.

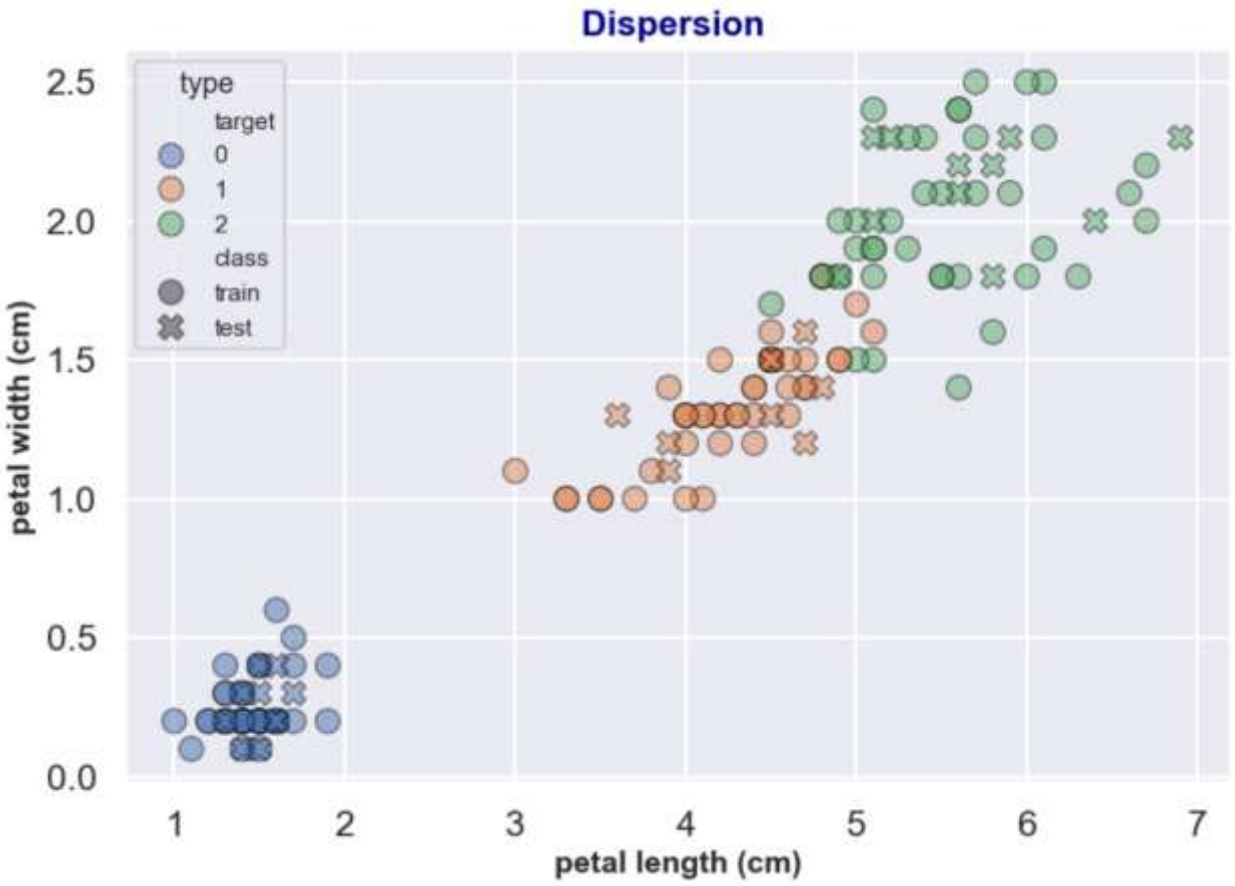
ویژگی های مدنظر در اینجا طول و عرض گلبرگ هستند بنابراین در متغیرهایی جداگانه آنها را ذخیره می کنیم.

سپس سایز نمودار ترسیمی را مشخص می کنیم و حال نمودار نقطه ای که نوعی نمودار دوبعدی است را تعریف میکنیم.

برای انکه جزئیات ترسیمی را بتوان اعمال کرد از آپشن های متفاوت این تابع استفاده می کنیم. ابتدا دیتای مدنظر که نمودار باید براساس آن رسم شود را مشخص می کنیم و ایکس و ایگرگ نمودار را تعریف می کنیم (همان متغیرهایی که ابتدا تعریف کردیم). آپشن hue درواقع بیان میکند که تفاوت رنگی برا جه اساسی باشد که ما در اینجا براساس ستون تارگت ها تفاوت رنگبندی را اعمال کرده ایم. سپس شکل نقطه را با توجه به ستون class تعریف می کنیم ینی باتوجه به اینکه نقطه ما ترین است یا تست نقطه ما دایره ای خواهد بود یا ضربدری و سپس سایز نقاط را مشخص کرده و از یک پالت رنگی با کنتراست بالا استفاده می کنیم تا تشخیص بهتر انجام شود. لبه ها را هم مشخص می کنیم تا تفکیک نقاط امکان پذیر باشد. شفافیت ها را هم کمتر از ۱ تنظیم می کنیم تا همپوشانی ها مشخص باشند و داده ها از دست نروند.

درنهایت هم نمودار را با تعیین کردن لیبل محور های ایکس و ایگرگ و نراهنمای نمودار اینجا ضروری است چون رنگ ها و شکل ها معنی دارند پس با نمایش آن به اتمام میرسانیم.

خروجی:



۲. سه بعدی:

کد:

```
from mpl_toolkits.mplot3d import Axes3D

#initializing the features
feature_x = "sepal length (cm)"
feature_y = "sepal width (cm)"
feature_z = "petal length (cm)"


sns.set_style("whitegrid")
sns.set_context("talk")

#seperating the classes
colors = np.array(["red", "blue", "green"])
labels = ["Setosa", "Versicolor", "Virginica"]


fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

for i in range(3):
    subset = total_df[total_df["target"] == i]
    ax.scatter(
        subset[feature_x], subset[feature_y], subset[feature_z],
        color=colors[i], label=labels[i], s=80, edgecolor="black", alpha=0.5)


ax.set_xlabel("Sepal length(cm)", fontsize=12, fontweight="bold")
ax.set_ylabel("Sepal width(cm)", fontsize=12, fontweight="bold")
ax.set_zlabel("petal length(cm)", fontsize=12, fontweight="bold")
ax.set_title("Dispersion", fontsize=14, fontweight="bold", color="darkblue")


ax.legend(title="Classes", fontsize=12, title_fontsize="13", loc="upper left")
plt.show()
```

برای ترسیم یک نمودار سه بعدی از from mpl_toolkits.mplot3d import Axes3D استفاده می کنیم.

سـپس سـه محور را تعریف می کنیم درواقع سـه تا از فیچر ها را مشـخص کرده و در متغیرهای x,y,z ذخیره میکنیم. مجددا برای جلوه بصـری بهتر اسـتایل را مشـخص کرده و فونت را نیز مشـخص میکنیم. برای تفکیک رنگ های کلاس های متفاوت آرایه ای از اسم رنگ ها تعریف می کنیم تا در آپشن color هنگام رسم نمودار استفاده کنیم. کلاس های متفاوت را هم که نام انواع گل زنبق است را لیبل میزنیم تا در label هنگام رسم استفاده کنیم.

در ادامه کد به تعیین سایز نمودار پرداخته و سپس ax را به تعیین می کنیم لازم به ذکر است که:

111 ⍰به این معنی است که یک ساب‌پلات در موقعیت **تنها پنل موجود** ایجاد شود .

projection='3d' ⍰به **Matplotlib**اطلاع می‌دهد که این نمودار باید **سه‌بعدی** باشد.

درادامه در حلقه for رنج تکرار را تا ۳ می گذاریم چون ۳ نوع کلاس داریم. سپس به رسم می پردازد.

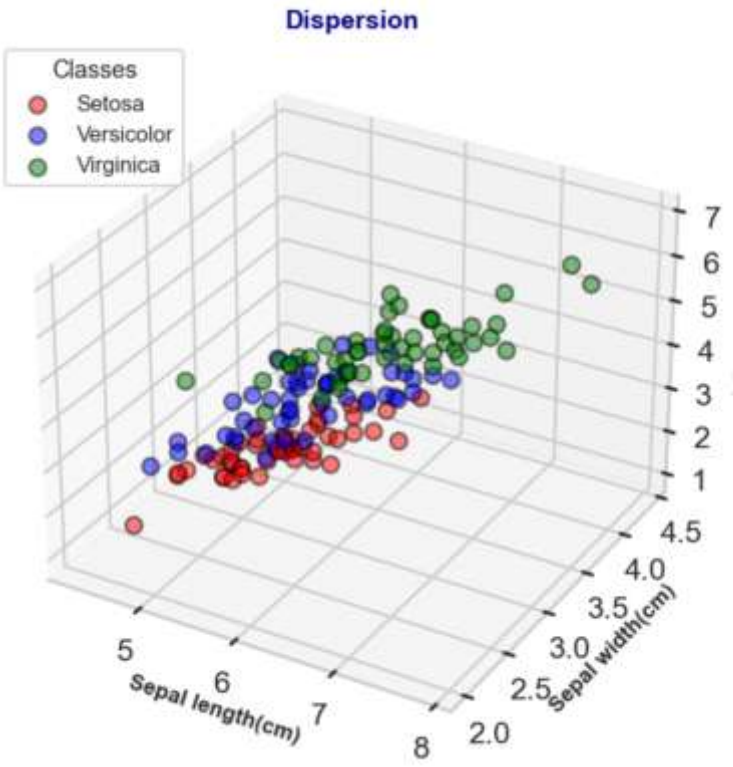
Subset ها زیرمجموعه های داده اصلی هستند و به ستون های مشخصی از دیتافر یم برمیگردد. بنابراین برای تعیین نمودار باید از آنها استفاده کنیم و سپس سایز و رنگ و لیبل ها را تعیین می کنیم (باتوجه به آن آرایه هایی که تعیین کرده بودیم) و شفافیت را هم کم تنظیم می کنیم تا بتوان همپوشانی ها را هم مشاهده کرد.

سپس لیبل محورهای ایکس و ایگرگ و زد را مشخص کرده و عنوان را تنظیم می کنیم.

راهنمای نمودار قطعا لازم است چون تفاوت لیبل ها و رنگ ها زیاد است و از لحاظ بصری بهتر است که راهنما را در بالا سمت چپ داشته باشیم. (location)

سپس نمودار رسم می شود.

خروجی:



۳. نمودار حرارتی:

کد:

```
correlation_matrix = total_df.drop(columns=["target", "class"]).corr()

plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5,
    cbar=True )

plt.title("Iris heatmap", fontsize=14, fontweight="bold", color="darkblue")
plt.show()
```

برای رسم نمودار حرارتی ابتدا ماتریس همبستگی را محاسبه کنیم ، سپس از آن برای ترسیم heatmap استفاده می کنیم

ابتدا از دیتافریم اصلی که تولید کرده بودیم باید با استفاده از دستور drop دو تا از ستون ها را حذف کنیم و سپس کورلیشن را محاسبه کنیم.

درنتیه یک **ماتریس عددی** ایجاد می‌شود که نشان می‌دهد هر ویژگی چقدر با بقیه ارتباط دارد .**اعداد بین 1-تا 1هستند:**

- :1+همبستگی مثبت کامل (مثلاً اگر Aزیاد شود، Bهم زیاد می‌شود.(
- :1-همبستگی منفی کامل (اگر Aزیاد شود، Bکم می‌شود.(
- :0هیچ رابطه‌ای بین ویژگی‌ها نیست.

پس از آن پلت رنگی را تعیین می کنیم

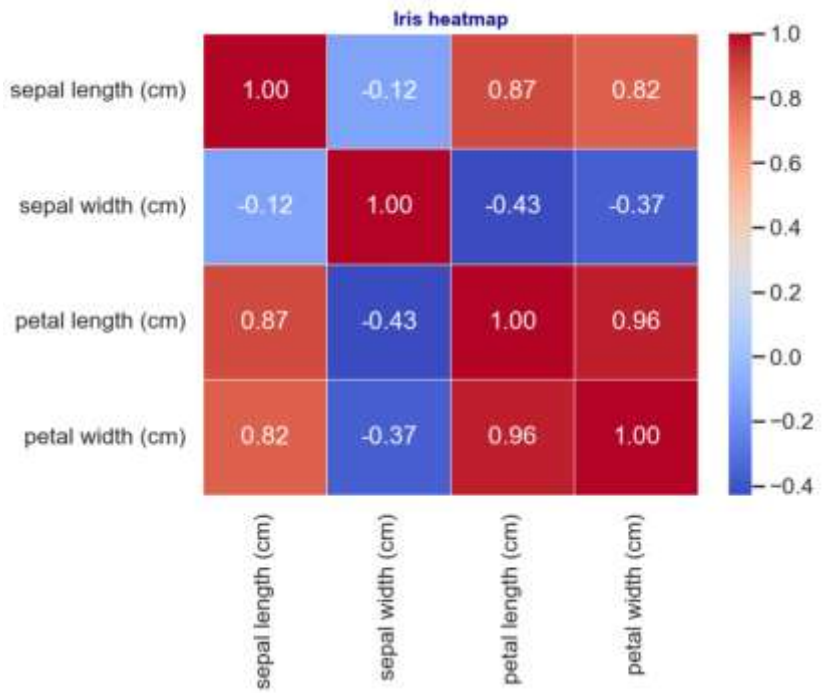
. دقت یا تعداد اعشاری که می خواهیم نمودار آن را نشان بدهد را نیز مشخص می کنیم.

Cbar درواقع تنالیته مورد استفاده کنار نمودار را مشخص میکند.

annot=Trueمقدار همبستگی را **به‌صورت عددی روی هر خانه‌ی جدول** نمایش می‌دهد.

درنهایت خروجی یک **نقشه رنگی (Heatmap)** ایجاد می‌شود که نشان می‌دهد **ویژگی‌های** دیتاست Iris چقدر به هم مرتبط هستند.

خروجی:



۴.PDF:

کد:

پس از اینکه اســتایل و فونت را مشــخص کردیم به این می پردازیم که با اســتفاده از subplotیک تصــویر ۲ در ۲ از نمودار ها تشکیل بدهیم و اندازه کل نمودار را نیز ۱۰ در ۱۲ تنظیم می کنیم.

لازم به ذکر است که axesیک **آرایه ۲×۲ از محورها** (Axes)برمی‌گرداند که هر نمودار روی یکی از آن‌ها رسم خواهد شد.

در حلقه for به هر ویژگی یک ایندکس نسبت داده می شود.

سپس در حلقه سطر و ستون را مشخص میکنیم درواقع مختصات سطری و ستونی را مشخص کرده تا اطلاعات در آن درج شوند:

- **نمودار چگالی احتمال (KDE)** را برای Trainو Testرسم می‌کند.
- data=total_df[total_df["class"] == "train"] **فقط داده‌های آموزش** انتخاب می‌شوند.
- data=total_df[total_df["class"] == "test"] **فقط داده‌های تست** انتخاب می‌شوند.
- x=feature محور Xمقدار ویژگی موردنظر است.
- fill=True → ناحیه‌ی زیر منحنی را **رنگی می‌کند.**
- color="blue" / "red" رنگ نمودار را برای **آبی** و **قرمز** مشخص می‌کند.
- ax=axes[row, col] مشخص می‌کند که **در کدام بخش از چیدمان ۲×۲** رسم شود.
- alpha=0.5 شفافیت رنگ را ۵۰٪ تعیین می‌کند که نواحی همپوشانی را مشخص می کنیم.
- plt.tight_layout() فاصله‌ی بین نمودارها را تنظیم می‌کند تا بهم نچسبند و بهتر دیده شوند.

```
sns.set_style("whitegrid")
sns.set_context("talk")
feature_names = iris_data.feature_names
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

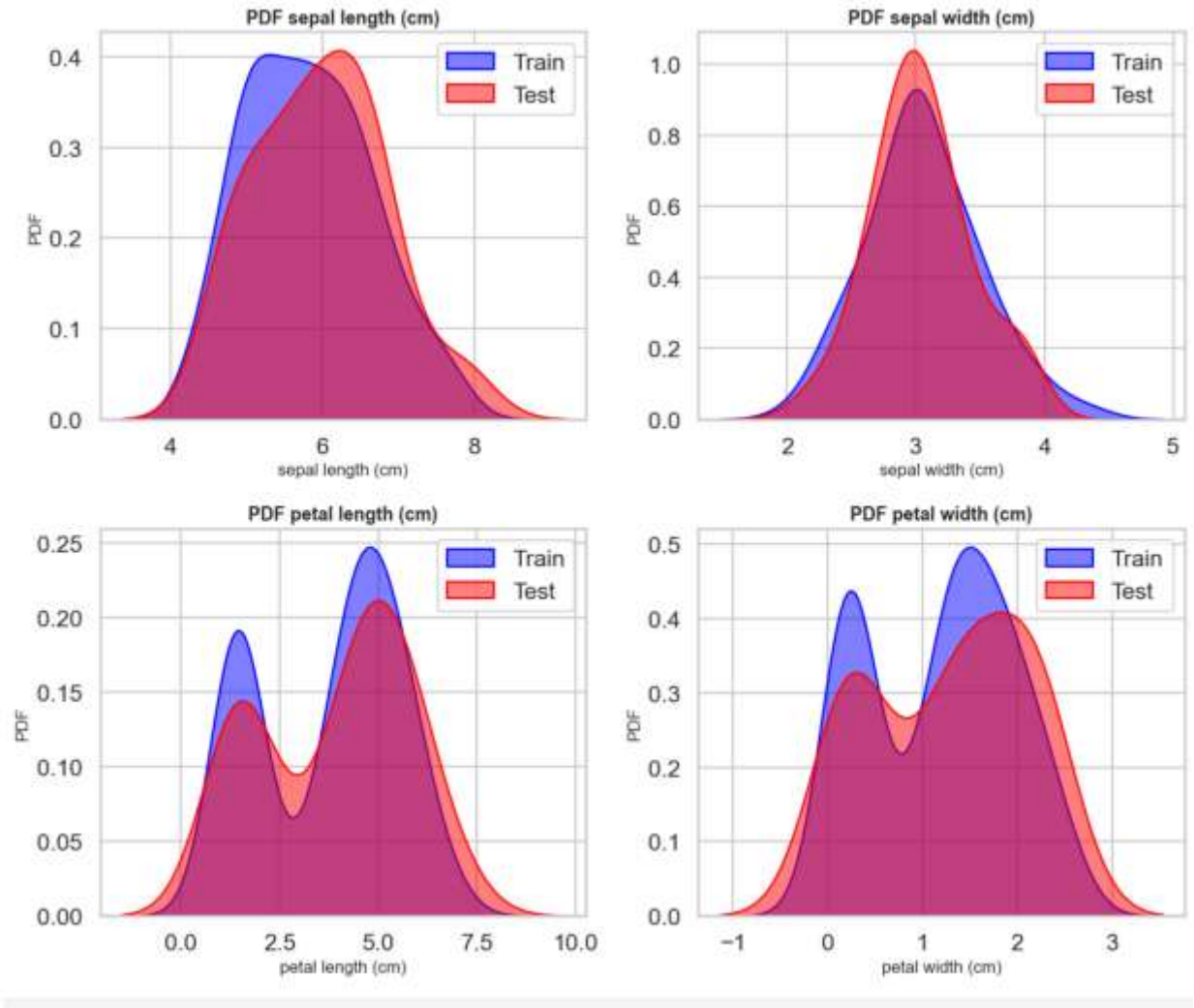

for i, feature in enumerate(feature_names):
    row, col = i // 2, i % 2

    sns.kdeplot(data=total_df[total_df["class"] == "train"], x=feature, label="Train", fill=True, color="blue", ax=axes[row, col], alpha=0.5)
    sns.kdeplot(data=total_df[total_df["class"] == "test"], x=feature, label="Test", fill=True, color="red", ax=axes[row, col], alpha=0.5)

    axes[row, col].set_title(f"PDF {feature}", fontsize=13, fontweight="bold")
    axes[row, col].set_xlabel(feature, fontsize=12)
    axes[row, col].set_ylabel("PDF", fontsize=12)
    axes[row, col].legend()

plt.tight_layout()
plt.show()
```

خروجی:


PDF sepal length (cm)
Train
Test
PDF
sepal length (cm)
PDF sepal width (cm)
Train
Test
PDF
sepal width (cm)
PDF petal length (cm)
Train
Test
PDF
petal length (cm)
PDF petal width (cm)
Train
Test
PDF
petal width (cm)

گسسته سازی پیوستگی

تحلیل آماری

کد:

```
setosa_df = total_df[total_df["target"] == 0]
setosa_stats = setosa_df.describe()
setosa_stats
```

در اینجا با استفاده از دیتافریم اصلی، دیتافریمی که تارگت انها مربوط به setosa بود را جدا کرده و در یک دیتافریم جدید ذخیره می کنیم سپس با استفاده از متد describe که اطلاعات آماری یک داده را به ما میدهد دیتافریم آماری را بدست می اوردیم.

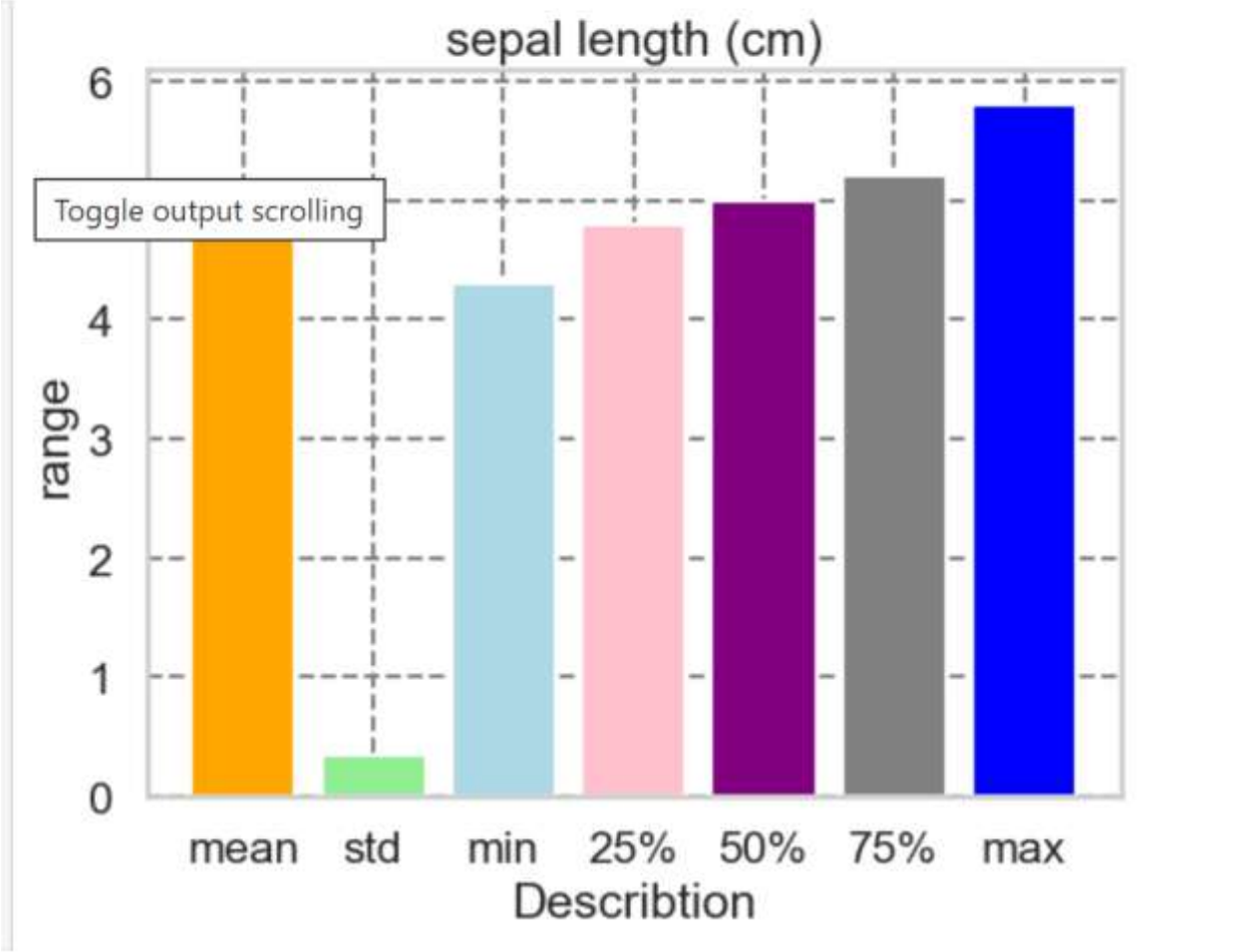
خروجی:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	50.00000	50.000000	50.000000	50.000000	50.0
mean	5.00600	3.428000	1.462000	0.246000	0.0
std	0.35249	0.379064	0.173664	0.105386	0.0
min	4.30000	2.300000	1.000000	0.100000	0.0
25%	4.80000	3.200000	1.400000	0.200000	0.0
50%	5.00000	3.400000	1.500000	0.200000	0.0
75%	5.20000	3.675000	1.575000	0.300000	0.0
max	5.80000	4.400000	1.900000	0.600000	0.0

رسم نمودار برای این خروجی:

در ادامه با استفاده از نمودار میله ای به طور مثال اولین فیچر که sepal length است را به صورت نمودار نشان می دهیم برای اینکار ابتدا باید index ها را جدا کنیم و سپس برای این ستون نمودار را تعریف کنیم برای آنکه هر یک از موارد آماری تفکیک باشند با یک رنگ آنرا رسم می کنیم و رنگ ها را نیز در ابتدا در آرایه تعریف می کنیم.

```
setosa_stats_index=setosa_stats.index
color=['orange','lightgreen','lightblue','pink','purple','gray','blue']
plt.bar(setosa_stats_index[1:],setosa_stats['sepal length (cm)'][1:], color=color)
plt.title('sepal length (cm)',loc='center')
plt.xlabel('Describtion')
plt.ylabel('range')
plt.grid(linestyle='--', color='gray')
plt.show()
```



می توانیم با استفاده از subplot برای همه ویژگی ها این نمودار ها را رسم کرده و مقایسه کنیم.